LLM-Based Semantic Extraction for Design-Time Business Process Compliance Checking

Yingchao Huo, Xiaoqing Chen, Jiahao Ma, Guosheng Kang✉, Wen Li, Jiayan Xiang
[1]Hunan Provincial Key Lab. for Services Computing and Novel Software Technology, HNUST, Xiangtan, Hunan, China
[2]School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan, China
Email: {huoyingchao666, xiaoqingchen696, jiahaoma2627, guoshengkang, wenli0023cs, jiayanxiang02}@gmail.com

***Abstract*—Business process compliance (BPC) checking aims to determine whether business processes conform to regulatory requirements and is an important task in business process management. Existing approaches typically transform natural-language regulations into structured representations for subsequent compliance checking. However, semantic extraction in this process often relies on manually designed rules or lexical resources, which requires considerable human effort and may have difficulty handling diverse regulatory expressions. To address this issue, we introduce LLM-based Semantic Extraction (LLM-SE) into a three-part design-time BPC checking framework. Given regulatory requirements, LLM-SE extracts six types of regulatory semantics, including modality, actor, action, condition, constraint, and exception. A deterministic post-processing procedure further performs structural adaptation, evidence alignment, and validation to construct structured Rule Records. The resulting Rule Records are combined with Process Records derived from BPMN models in Part 3, which follows the matching and three violation definitions of Sun et al. Experiments on regulatory requirements and process models demonstrate the effectiveness of the proposed approach.**

***Index Terms*—Business Process Compliance, Design-Time Compliance Checking, Large Language Models, Compliance Requirement Parsing, Semantic Extraction**

I. Introduction

Business process compliance (BPC) aims to ensure that business processes conform to laws, regulations, standards, and organizational policies [1]. Design-time compliance checking identifies potential violations before execution, reducing non-compliance risks and costs. A key challenge is the semantic gap between natural-language regulatory requirements and the activities, responsibilities, and control-flow structures of process models. Existing studies have explored compliance rule representations to support automated checking [2].

Early BPC studies used structured compliance representations. Awad et al. [3] combined BPMN-Q with temporal logic, Elgammal et al. [4] introduced compliance patterns, and Governatori et al. [5] used LegalRuleML for compliance checking. Such representations still need to be derived from regulatory documents. Sleimi et al. [6] extracted semantic legal metadata using natural language processing, while Michel et al. [7] identified decision rules from legislation. Winter et al. [8] assessed process models against regulatory documents. Sun et al. [9] proposed a multi-granularity approach that extracts six regulatory semantics—modality, actor, action, condition, constraint, and exception—for violation detection using process-model information. Their approach reduces manual formalization but still relies on predefined linguistic patterns and lexical resources for semantic extraction.

Large language models (LLMs) have also been applied to regulatory analysis. De Kinderen and Winter [10] studied prompt-based generation of legal requirement models. Barrientos et al. [11] combined LLM-based and algorithmic methods to analyze the impact of regulatory changes on BPC, and later studied legal-change management [12]. Guliani et al. [13] proposed De Jure, an iterative LLM-based pipeline for structured regulatory rule extraction. For process-related tasks, Wang et al. [14] used LLM-generated process information for compliance risk identification, Park et al. [15] extracted Declare constraints from natural-language process descriptions, and Kölbel et al. [16] combined BPMN models with external security knowledge using LLMs for process explanation and compliance assessment.

However, these studies differ in their representations and checking tasks. We therefore investigate two questions: whether LLMs can extract the semantics required by an existing design-time BPC framework more accurately than a conventional extractor under the same fine-grained representation and evaluation procedure, and how the resulting Rule Records affect downstream compliance checking.

We investigate LLM-SE within a three-part design-time BPC framework: Process Model Parsing, Compliance Requirement Parsing, and Compliance Checking. The first two parts independently transform BPMN models and natural-language requirements into Process Records and Rule Records, respectively. Our contribution focuses on Compliance Requirement Parsing, where LLM-SE extracts regulatory semantics and deterministic post-processing constructs valid Rule Records. Compliance Checking combines both records for rule–process matching and detection of missing-action, incorrect-actor, and out-of-order violations.

The main contributions of this work are:

- We propose LLM-based Semantic Extraction (LLM-SE) for compliance requirement parsing. It extracts six regulatory semantic elements—modality, actor, action, condition, constraint, and exception—and constructs structured Rule Records for compliance checking.
- We design a modular prompt combining task and schema specifications with semantic examples, semantic guidance, and output-discipline instructions. We also de-

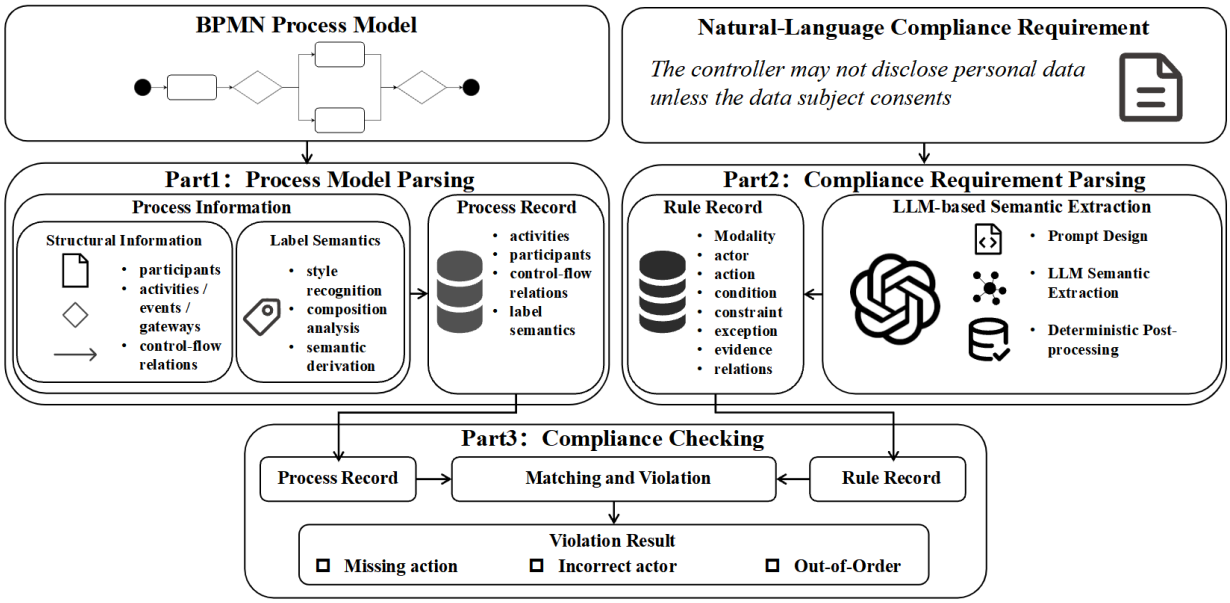


Fig. 1: Overall framework for design-time business process compliance checking.

velop deterministic post-processing that performs structural adaptation, evidence alignment, relation cleanup, and validation to convert raw LLM outputs into valid Rule Records.

- We evaluate LLM-SE against an independently reconstructed Rules-Only baseline under the same semantic representation and evaluation procedure. We further analyze prompt components and examine how the resulting Rule Records affect compliance checking for three violation types.

II. Problem Formulation and Framework

A. Problem Formulation

Following the design-time BPC setting of Sun et al. [9], we aim to identify potential violations of natural-language compliance requirements in BPMN process models before execution.

A BPMN model is transformed into a Process Record containing process elements, participants, and control-flow relations, with actors, actions, and business objects derived from process labels and associated with their activities. A compliance requirement is transformed into a Rule Record containing six regulatory semantic elements: modality, actor, action, condition, constraint, and exception.

Following Sun et al. [9], modality specifies the normative type, such as obligation, prohibition, or permission. Actor identifies the responsible entity, and action describes the required behavior. Condition specifies when a requirement applies, constraint describes restrictions on its applicability, and exception specifies when it does not apply. The Rule Record also preserves supporting textual evidence and relations between elements.

The two records support rule–process matching and checking for three violation types: missing action, incorrect actor, and out-of-order execution.

B. Overall Framework

As shown in Fig. 1, Process Model Parsing extracts structures, control-flow relations, and label semantics from BPMN models to form Process Records. Independently, Compliance Requirement Parsing uses LLM-SE and deterministic post-processing to construct valid Rule Records from natural-language requirements. Compliance Checking combines both records for rule–process matching and detection of missing-action, incorrect-actor, and out-of-order violations.

III. Methodology

A. Process Model Parsing

Following Sun et al. [9], we extract structural and semantic information from BPMN models. A deterministic parser identifies activities, events, gateways, pools, lanes, sequence flows, and their labels. Control-flow relations, including direct connections, reachability, and activity ordering, are derived from sequence flows.

Actors are identified from process context, such as pools and lanes. Following Sun et al., activity labels undergo label style recognition, label composition analysis, and semantic content derivation to identify actions and business objects. The resulting Process Record contains structural information and associates extracted label semantics with their corresponding activities for compliance checking.

B. Compliance Requirement Parsing

Unlike Sun et al. [9], whose semantic extraction relies on predefined linguistic patterns and lexical resources, our LLM-based Semantic Extraction (LLM-SE) uses prompt-guided ex-

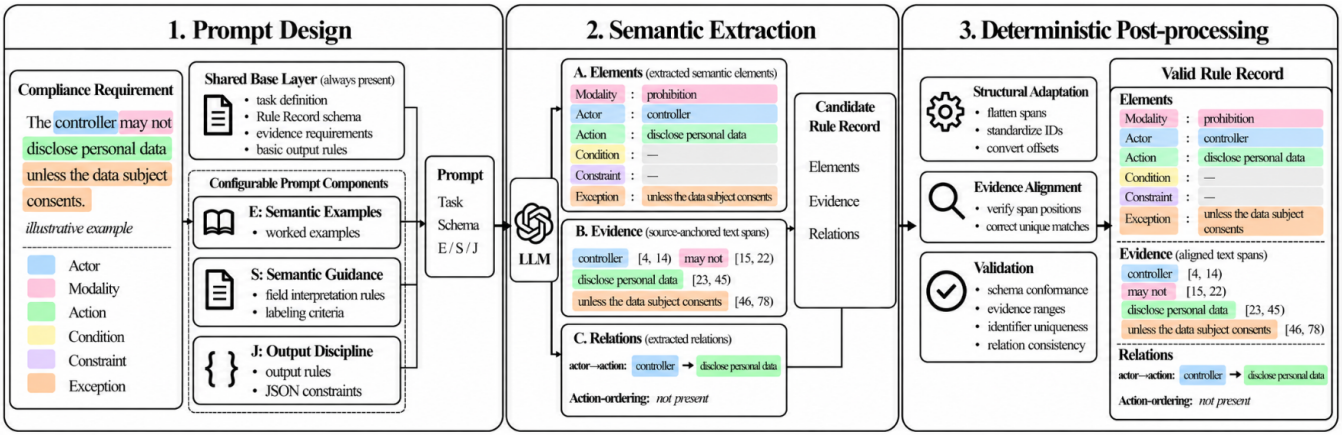


Fig. 2: Detailed workflow of LLM-based semantic extraction.

traction to identify regulatory semantics from natural-language requirements. We retain the same six semantic categories—modality, actor, action, condition, constraint, and exception—for comparison under a common representation. As shown in Fig. 2, LLM-SE consists of three steps: Prompt Design, Semantic Extraction, and Deterministic Post-processing.

Prompt Design. We design a modular prompt with a fixed shared base layer and three configurable components. The base layer defines the extraction task, target Rule Record schema, evidence requirements, and basic output rules. Component E provides worked examples, S provides detailed semantic guidance, and J adds JSON formatting and output-discipline instructions. This design separates essential extraction requirements from optional guidance, allowing the effects of E, S, and J to be evaluated independently without changing the basic task or output structure.

LLM-based Semantic Extraction. Given a compliance requirement and the designed prompt, the LLM extracts semantic elements, supporting textual evidence, and relations between elements in a single call. The output forms a candidate Rule Record in JSON format, with source-anchored evidence spans and element identifiers for actor–action and ordering relations. For example, from "The controller may not disclose personal data unless the data subject consents," the model identifies "controller" as the actor, "disclose personal data" as the action, and "unless the data subject consents" as an exception. Unlike the Rules-Only baseline, LLM-SE does not use predefined linguistic extraction patterns or baseline predictions to identify these elements.

Deterministic Post-processing. Following the use of structured outputs and schema validation in regulatory analysis [11], we apply deterministic processing to adapt and validate the generated Rule Record. Structural adaptation converts supported output formats into the target schema by flattening nested spans, standardizing identifiers, converting valid numeric strings into integer offsets, and generating normalized values from the extracted text.

Evidence alignment checks whether each extracted span matches its recorded source position. Character offsets use zero-based start and exclusive end positions. Correct coordinates are retained; otherwise, the procedure searches for an exact text match within the corresponding source clause. A unique match allows coordinate correction, while unmatched or ambiguous spans are removed. For example, if "disclose" is extracted with incorrect offsets but occurs only once in the clause, its position can be recovered without changing the extracted text.

Relations referring to removed actors or actions are then discarded. Final validation checks schema conformance, identifier uniqueness, evidence ranges, clause containment, and relation consistency. Records that fail validation are rejected rather than treated as valid Rule Records. Unlike semantic extraction, this procedure repairs structural and evidence-position errors without inferring missing elements or changing their semantic categories.

C. Compliance Checking

Part 3 combines the Process Record and Rule Record using the rule–process matching and three violation definitions of Sun et al. [9]. Our contribution concerns Part 2; the checking formulas are retained below.

For rule r, A_r, P_r, and U_r denote its actions, actors, and sequential constraints. For process model m, A_m, E_m, and R_m denote activities, events, and actors. The functions $action$ and bs_obj extract actions and business objects from activity and event labels. The mappings f_r and f_m associate actors or business objects with their corresponding actions, and F_m denotes the process control-flow relation.

We compute text similarity $sim(x, y)$ using cosine similarity between MPNet-based Sentence-BERT embeddings [17, 18]. Thresholds τ, γ, and ϑ control rule–process matching, action matching, and actor comparison, respectively; ϑ corresponds to θ in the experimental results.

Rule–Process Matching. The set $D_{r,m}$ pairs each rule action with its most similar element in $action(A_m \cup E_m)$. Similarly, $O_{r,m}$ pairs each rule actor with its most similar

element in $bs_obj(A_m \cup E_m) \cup R_m$. Sun's matching score is

$$\mathcal{F}(r, m, \tau) = \max\left(\frac{\sum_{(a_r, a_m) \in D_{r,m}} sim(a_r, a_m) > \tau}{|D_{r,m}|}, \frac{\sum_{(o_r, o_m) \in O_{r,m}} sim(o_r, o_m) > \tau}{|O_{r,m}|} \right). \tag{1}$$

The sums count mapping pairs whose similarity exceeds τ.

Missing-Action Violation. The score counts mapped rule actions with similarity below γ:

$$action_violation(r, m, \gamma) = \frac{\left|\{(a_r, a_m) \in D_{r,m} \mid sim(a_r, a_m) < \gamma\}\right|}{|A_r|}. \tag{2}$$

Incorrect-Actor Violation. Rule actors with matched actions form $R_{r,m,\gamma}$, and the corresponding shared process-side candidates form $C_{r,m,\gamma}$:

$$R_{r,m,\gamma} = \{g \in P_r \mid \exists (a_r, a_m) \in D_{r,m}, f_r(g) = a_r \wedge sim(a_r, a_m) > \gamma\}, \tag{3}$$

$$C_{r,m,\gamma} = \{r \in bs_obj(A_m \cup E_m) \cup R_m \mid \exists g \in R_{r,m,\gamma}, sim(f_r(g), f_m(r)) > \gamma\}. \tag{4}$$

A rule actor contributes to the score if a candidate in the shared set has similarity below ϑ:

$$actor_violation(r, m, \gamma, \vartheta) = \frac{\left|\{r \in R_{r,m,\gamma} \mid \exists r' \in C_{r,m,\gamma}, sim(r, r') < \vartheta\}\right|}{|R_{r,m,\gamma}|}. \tag{5}$$

Out-of-Order Violation. Sequential constraints with both endpoint actions matched form

$$U_{r,m,\gamma} = \{(u_r, u'_r) \in U_r \mid \exists\{(u_r, u_m) \in D_{r,m}, (u'_r, u'_m) \in D_{r,m}\}, sim(u_r, u_m) > \gamma \wedge sim(u'_r, u'_m) > \gamma\}. \tag{6}$$

The score is the fraction of evaluable constraints that do not satisfy the prescribed order:

$$order_violation(r, m, \gamma) = \frac{|U_{r,m,\gamma}| - \left|\{(u_r, u'_r) \in U_{r,m,\gamma} \mid \exists\{(u_r, u_m) \in D_{r,m}, (u'_r, u'_m) \in D_{r,m}\}, (u_m, u'_m) \in F_m \wedge (u'_m, u_m) \notin F_m\}\right|}{|U_{r,m,\gamma}|}. \tag{7}$$

IV. Experiments and Results

A. Experimental Setup

Datasets. For semantic extraction, we construct EStG-150 using 150 sentences from the publicly available Austrian Income Tax Act, following Sun et al. [9]. Reference annotations for all six semantic elements were manually reviewed, corrected, and finalized by the authors; LLM-generated annotations served only as initial candidates.

For compliance checking, we construct a GDPR-based benchmark with 115 BPMN process cases and 33 evaluated compliance requirements. The source material consists of nine GDPR rule texts divided into 74 sentences. The benchmark contains 201 applicable process–requirement–violation-type units (113 missing-action, 80 incorrect-actor, and 8 out-of-order), including 69 violations and 132 compliant units. Because this benchmark was also used for development and parameter selection, the results do not represent performance on an independent test set.

Compared Methods. For semantic extraction, we compare LLM-SE with a locally reconstructed rule-based approach from Sun et al. [9]. Sun et al. uses syntactic analysis and manually designed linguistic rules, whereas LLM-SE uses an LLM with deterministic post-processing. Both use the same input texts, reference annotations, Rule Record representation, and evaluation procedure.

For compliance checking, we compare LLM-SE with local reconstructions of Sun et al. [9] and Winter et al. [8], implemented independently from their published methods. Sun et al. and LLM-SE share the same compliance checker, differing mainly in regulatory semantic extraction, while Winter et al. uses a separate checking procedure. All three methods use the same reference labels and evaluation metrics.

Model and Parameter Settings. LLM-SE uses DeepSeek-V4-Pro-0813 with temperature 0, top-p 1.0, and a maximum output length of 4096 tokens. The complete prompt combines the shared base layer with the E, S, and J components described in Section III.

For compliance checking, Sun et al. and LLM-SE share the same similarity measure and violation detection procedure, using thresholds $\tau = 0.8$, $\gamma = 0.55$, and $\theta = 0.45$. Winter et al. uses a separate procedure with $\gamma = 0.4$ and $\delta = 0.8$.

Evaluation Metrics. For semantic extraction, a predicted phrase is correct when its character span overlaps a reference span of the same semantic type. We report F1 for actor, action, condition, constraint, and exception. Overall Micro-F1 pools true positives (TP), false positives (FP), and false negatives (FN) across these five elements. Modality uses four-class Macro-F1 and is excluded from Overall Micro-F1.

For compliance checking, we report Precision, Recall, and F1 across the three violation types. For a reference violation, a prediction of *violated* is TP, while *satisfied* or *unknown* is FN. For a reference-compliant unit, *violated* is FP and *satisfied* is TN; *unknown* is not counted as TN. Overall metrics are computed from pooled TP, FP, and FN across all evaluated units.

B. Semantic Extraction Results

Table I shows that LLM-SE improves Overall Micro-F1 from 0.763 to 0.838 (7.5 percentage points). It outperforms Sun et al. on modality, action, condition, and constraint, with the largest gain in constraint (0.618 to 0.743), but performs worse on actor and exception.

TABLE I: Semantic extraction performance of Sun et al. and LLM-SE.

Element	Sun et al.	LLM-SE (Ours)
Modality (Macro-F1)	0.713	**0.769**
Actor	**0.820**	0.758
Action	0.893	**0.944**
Condition	0.774	**0.838**
Constraint	0.618	**0.743**
Exception	**0.880**	0.762
Overall (Micro-F1)*	0.763	**0.838**

*Pooled over actor, action, condition, constraint, and exception; modality is excluded.

TABLE II: Ablation results of prompt components.

E	S	J	Micro-F1
−	−	−	0.7380
−	−	+	0.7261
−	+	−	0.8107
−	+	+	0.8142
+	−	−	**0.8354**
+	−	+	0.8299
+	+	−	0.8268
+	+	+	0.8224

C. Prompt Component Analysis

Table II compares examples (E), semantic guidance (S), and output constraints (J); "+" and "-" indicate enabled and disabled components. Micro-F1 pools the five non-modality elements.

E improves F1 in all matched configurations. S helps mainly without E, while J has variable effects. E alone achieves the highest F1 (0.8354), compared with 0.8224 for E/S/J, showing that more instructions do not necessarily help. The shared base prompt is fixed, and ablation results come from a separate run.

D. Compliance Checking Results

Threshold Sensitivity. Fig. 3 examines the effects of action (γ) and actor (θ) similarity thresholds on compliance-checking F1. With $\theta = 0.45$, LLM-SE reaches its highest F1 of 0.5341 at $\gamma = 0.55$, while Sun et al. peaks at 0.4706 at $\gamma = 0.45$. LLM-SE outperforms Sun et al. across all tested values of γ. With $\gamma = 0.55$, LLM-SE performs best at $\theta = 0.45$, while Sun et al. remains unaffected by θ.

Performance Comparison. On the GDPR-based benchmark (Table III), LLM-SE achieves an F1 of 0.5341 versus 0.4458 for Sun et al. under shared thresholds, an improvement of 8.83 percentage points. It also outperforms Winter et al. (0.5033) by 3.08 percentage points. LLM-SE achieves the highest recall (0.6812), while Winter et al. achieves the highest precision (0.4634). This suggests that improved semantic extraction benefits downstream compliance checking.

Despite these improvements, LLM-SE still produces false-positive violations. Accurate semantic extraction alone does not guarantee reliable matching between regulatory requirements and BPMN process elements, which remains a challenge.

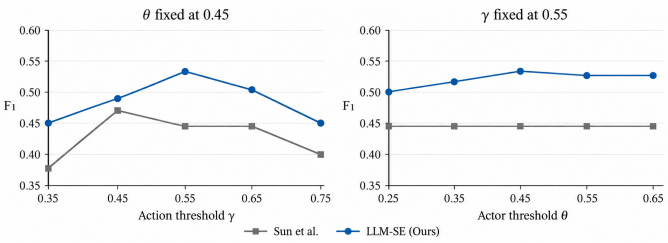


Fig. 3: Effects of action and actor similarity thresholds on compliance-checking F1.

TABLE III: Comparison of compliance-checking performance.

Method	Precision	Recall	F1
Sun et al.	0.3814	0.5362	0.4458
Winter et al.	**0.4634**	0.5507	0.5033
LLM-SE (Ours)	0.4393	**0.6812**	**0.5341**

V. Conclusion

We propose LLM-SE for design-time business process compliance checking. It combines prompt-based extraction with deterministic post-processing to construct Rule Records containing six regulatory semantic elements, reducing reliance on manually designed extraction rules and lexical resources. These records support compliance checking against BPMN process models.

On EStG-150, LLM-SE improves Overall Micro-F1 from 0.763 to 0.838 compared with the Sun-based method. Prompt analysis shows consistent benefits from semantic examples, while additional instructions do not always improve performance. On the GDPR-based development benchmark, LLM-SE achieves a checking F1 of 0.5341, outperforming the evaluated Sun and Winter baselines. However, reliable matching between regulatory semantics and BPMN process elements remains challenging. Future work will improve this matching and evaluate the approach on independent datasets.

Acknowledgment

This work is partially supported by the Hunan Provincial Natural Science Foundation of China under Grant No: 2026JJ50519. *Guosheng Kang* is the corresponding author.

References

[1] M. Hashmi, G. Governatori, H.-P. Lam, and M. T. Wynn, "Are we done with business process compliance: state of the art and challenges ahead," *Knowledge and Information Systems*, vol. 57, no. 1, pp. 79–133, 2018.

[2] A. Zasada, M. Hashmi, M. Fellmann, and D. Knuplesch, "Evaluation of compliance rule languages for modelling regulatory compliance requirements," *Software*, vol. 2, no. 1, pp. 71–120, 2023.

[3] A. Awad, G. Decker, and M. Weske, "Efficient compliance checking using BPMN-Q and temporal logic," in *Business Process Management*. Springer, 2008, pp. 326–341.

[4] A. Elgammal, O. Turetken, W.-J. van den Heuvel, and M. Papazoglou, "Formalizing and appling compliance patterns for business process compliance," *Software and Systems Modeling*, vol. 15, no. 1, pp. 119–146, 2016.

[5] G. Governatori, M. Hashmi, H.-P. Lam, S. Villata, and M. Palmirani, "Semantic business process regulatory compliance checking using LegalRuleML," in *Knowledge Engineering and Knowledge Management*. Springer, 2016, pp. 746–761.

[6] A. Sleimi, N. Sannier, M. Sabetzadeh, L. Briand, and J. Dann, "Automated extraction of semantic legal metadata using natural language processing," in *2018 IEEE 26th International Requirements Engineering Conference (RE)*. IEEE, 2018, pp. 124–135.

[7] M. Michel, D. Djurica, and J. Mendling, "Identification of decision rules from legislative documents using machine learning and natural language processing," in *Proceedings of the 55th Hawaii International Conference on System Sciences*, 2022, pp. 6247–6256. [Online]. Available: https://hdl.handle.net/10125/80097

[8] K. Winter, H. van der Aa, S. Rinderle-Ma, and M. Weidlich, "Assessing the compliance of business process models with regulatory documents," in *Conceptual Modeling*. Springer, 2020, pp. 189–203.

[9] X. Sun, S. Yang, C. Zhao, and D. Yu, "Design-time business process compliance assessment based on multi-granularity semantic information," *The Journal of Supercomputing*, vol. 80, no. 4, pp. 4943–4971, 2024.

[10] S. de Kinderen and K. Winter, "Towards taming large language models with prompt templates for legal GRL modeling," in *Enterprise, Business-Process and Information Systems Modeling*. Springer, 2024, pp. 213–228.

[11] M. Barrientos, K. Winter, and S. Rinderle-Ma, "Impact analysis of regulatory requirement changes on business process compliance," *Information and Software Technology*, vol. 194, p. 108079, 2026.

[12] M. Barrientos, J. Loebbecke, K. Winter, and S. Rinderle-Ma, "Taming the complexity of legal change for business process compliance," *Business & Information Systems Engineering*, vol. 68, no. 5, pp. 1147–1172, 2026.

[13] K. Guliani, D. Gill, D. Landsman, N. Eshraghi, K. Kumar, and L. Gondara, "De Jure: Iterative LLM self-refinement for structured extraction of regulatory rules," *arXiv preprint arXiv:2604.02276*, 2026. [Online]. Available: https://arxiv.org/abs/2604.02276

[14] Y. Wang, N. Yi, W. Chang, and J. Yu, "Leveraging LLMs for automated compliance risk identification in business processes," in *Neural Information Processing*, ser. Communications in Computer and Information Science, vol. 2287. Springer, 2025, pp. 456–471.

[15] G. Park, J. Kofferath, and M. Cho, "Extracting declarative constraints for process modeling from natural language descriptions with large language models," *Decision Support Systems*, vol. 203, p. 114627, 2026.

[16] L. M. Kölbel, L. Poss, and S. Schönig, "Context is key for cybersecurity: leveraging external knowledge for process model explanation via LLMs," *International Journal of Information Security*, vol. 25, p. 108, 2026.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, 2019, pp. 3982–3992. [Online]. Available: https://aclanthology.org/D19-1410/

[18] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "MPNet: Masked and permuted pre-training for language understanding," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 16 857–16 867. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/c3a690be93aa602ee2dc0ccab5b7b67e-Abstract.html